

Assignment: 03

Scenario:

An AI classroom system tracks daily student activities and updates the activity log dynamically.

Task:

- Create a list of 5 classroom activities.
- Add one new activity at the end.
- Insert a new activity in the middle of the list.
- Remove one outdated activity.
- Remove the last activity and store it separately.
- Display:
 - Final activity list
 - Removed activity

Sample Input:

["Attendance", "Quiz", "Assignment", "Presentation", "Discussion"]

Sample Output:

Final Activities:

['Attendance', 'Quiz', 'Group Work', 'Presentation', 'Discussion']

Removed Activity:

Review

2. Event Participant Manager (List Operations)

Scenario:

An event management system maintains a list of participants and updates it dynamically.

Task:

- Create a list of participants.

- Add multiple new participants at once.
- Reverse the participant order.
- Sort participants alphabetically.
- Display list after each operation.

Sample Input:

```
["Ayaan", "Nila", "Rahim"]
```

```
["Sami", "Rafi", "Toma"]
```

Sample Output:

After Adding:

```
['Ayaan', 'Nila', 'Rahim', 'Sami', 'Rafi', 'Toma']
```

After Reverse:

```
['Toma', 'Rafi', 'Sami', 'Rahim', 'Nila', 'Ayaan']
```

After Sort:

```
['Ayaan', 'Nila', 'Rahim', 'Rafi', 'Sami', 'Toma']
```

3. Library Member System (Dictionary)

Scenario:

A library system stores member details and updates them as needed.

Task:

- Create a dictionary with member details (name, age, membership type).
- Add a new field (city).
- Update membership type.
- Remove age field.
- Check if “email” exists.
- Print:
 - All keys
 - All values

- Final dictionary

Sample Input:

```
{"name": "Karim", "age": 22, "membership": "Gold"}
```

Sample Output:

Keys: dict_keys(['name', 'membership', 'city'])

Values: dict_values(['Karim', 'Gold', 'Dhaka'])

Final Data:

```
{'name': 'Karim', 'membership': 'Gold', 'city': 'Dhaka'}
```

4. AI Survey Analyzer (Nested Dictionary + List)**Scenario:**

An AI system collects survey responses from multiple users.

Task:

- Create a list containing multiple user responses (each response is a dictionary).
- Access the second user response.
- Extract the user's name.
- Print part of the name (middle characters).

Sample Input:

```
[  
  {"name": "Sabrina", "age": 24},  
  {"name": "Mehtaz", "age": 30},  
  {"name": "Oishi", "age": 28}  
]
```

Sample Output:

Second User:

```
{'name': 'Mehtaz', 'age': 30}
```

Extracted Name:

eh

5. Smart Food Inventory System (Set)

Scenario:

A restaurant tracks available and newly arrived food items.

Task:

- Create two sets: available items and new delivery items.
- Find common items.
- Find all items combined.
- Find items only available in the first set.
- Check whether a specific item exists.

Sample Input:

Available: {"Rice", "Chicken", "Tea", "Coffee"}

New: {"Coffee", "Burger", "Juice"}

Sample Output:

Common Items:

{'Coffee'}

All Items:

{'Rice', 'Chicken', 'Tea', 'Coffee', 'Burger', 'Juice'}

Only Available:

{'Rice', 'Chicken', 'Tea'}

Check Coffee:

True